"מבצע מגש פיצה" 🍕🕶️

חלק ראשון

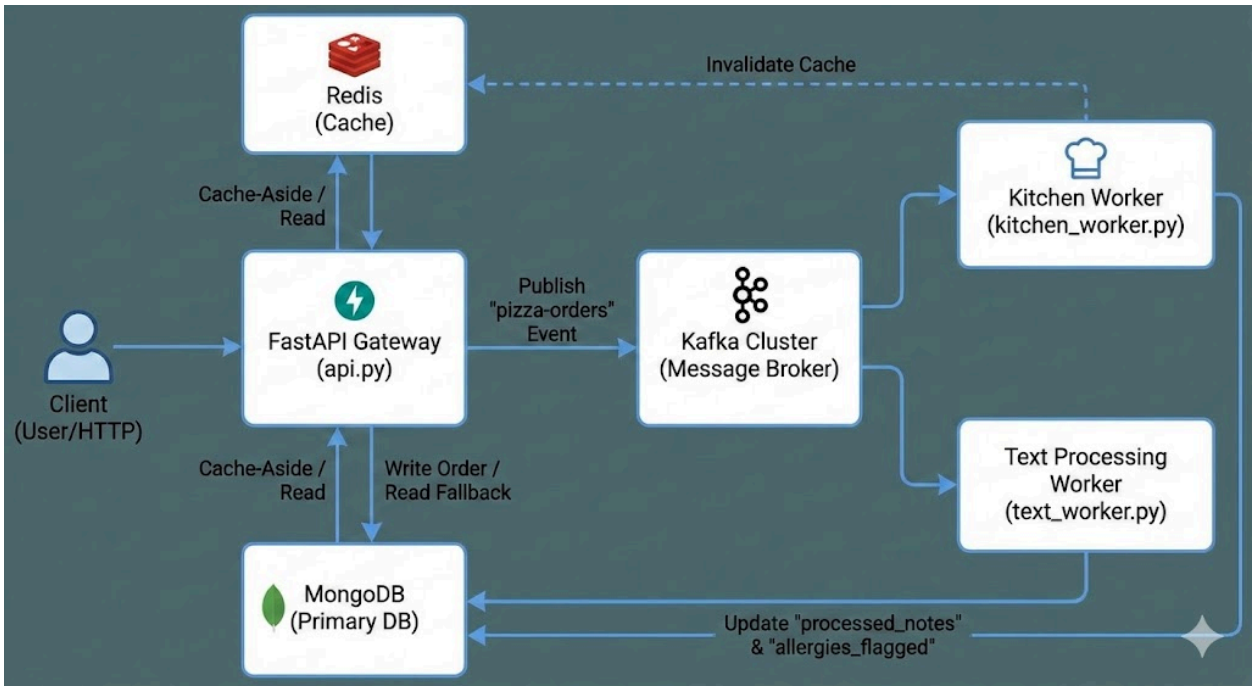
רקע

ברוכים הבאים ל"פיצריה". כלפי חוץ, אתם בונים מערכת ניהול הזמנות למסעדה. בפועל, ה"פיצריה" היא חזית (Front Operation) של סוכנות ביון. המערכת משמשת לקליטה, התממה (Sanitization) ואחסון של העברות נשק, ציוד טקטי ומסמכים מסווגים.

- **סוגי הפיצות (`pizza_type`)** הם למעשה קידוד לסוג המטען (למשל: "Margherita" = תחמושת 9 מ"מ, "Hawaiian" = כונן התקן USB מסווג).
- **ההוראות המיוחדות (`special_instructions`)** מכילות הערות שטח מוצפנות מהסוכנים. מילים תמימות כמו "אלרגיה" (Allergy) או "בוטנים" (Peanut) הן מילות קוד שמתריעות על חשד למכשירי מעקב (Trackers) או האזנה על המטען.

זרימת המידע (Data Flow):

1. סוכני שטח (הלקוחות) שולחים בקשה ל"פיצה ספציפית" דרך ה-FastAPI (API) בפורמט JSON.
2. ה-API שומר את הרשומה ב-**MongoDB** בסטטוס `PREPARING` .
3. ה-API מפרסם אירוע ל-**Kafka** אל ה-Topic שנקרא `pizza-orders` (כדי לשמור על סיפור הכיסוי).
4. **שני צוותים (Workers) פועלים במקביל על אותו אירוע:**
   - **צוות הלוגיסטיקה (Worker א' - "המטבח"):** קולט פיזית את המטען, מאבטח אותו בכספת (השהיה של 15 שניות), מעדכן את הסטטוס ב-Mongo ל-`DELIVERED`, ומוחק את הרשומה מ-**Redis** כדי לאלץ רענון נתונים.
   - **צוות המודיעין וההתממה (Worker ב' - "מעבד הטקסט"):** מנתח את הערות השטח. הוא מחפש מילות קוד מזהירות (כמו "peanut", "gluten"). אם נמצאו, הוא מדליק דגל אדום למכשירי מעקב הדורשים טיפול מיוחד. לאחר מכן, הוא מנקה את הטקסט מסימני פיסוק, הופך אותו לאותיות גדולות (ליצירת "פרוטוקול מבצעי" נקי) ומעדכן הכל בחשאי ב-Mongo.
5. כאשר החפ"ק בודק את סטטוס המטען, המערכת בודקת קודם ב-**Redis** (למהירות שיא). אם חסר (Cache Miss), שולפים מ-MongoDB, שומרים ב-Redis ל-60 שניות, ומחזירים תשובה.



הנחיות ושלבי ביצוע

שלב 1: תשתית ה-Docker (Docker Compose)

עליכם ליצור קובץ `docker-compose.yml` שיכלול את התשתיות:

- `kafka` (פורט 9092) - ערוץ התקשורת המוצפן שלנו.
- `mongo` (פורט 27017) - ארכיון המבצעים.
- `redis` (פורט 6379) - זיכרון טקטי לטווח קצר.
- `api` - שער הכניסה למערכת (FastAPI על פורט 8000).
- `kitchen_worker` - סקריפט הפייתון שמדמה את קליטת המטען (ה"מטבח").
- `text_worker` - סקריפט הפייתון שאחראי על התממת המידע ("ניתוח הטקסט").

שלב 2: פיתוח ה-FastAPI (API Gateway)

צרו קובץ `api.py` הכולל מודל Pydantic שיודע לקבל קובץ של הזמנות JSON עם השדות: `order_id`, `pizza_type`, `size`, `quantity`, `is_delivery`, `special_instructions`.

1. **`/POST /uploadfile`**
   - קולט את רשימת ההזמנות מתוך קובץ שנשלח ל-endpoint (מערך JSON).
   - שומר ב-Mongo כל הזמנה עם שדה סטטוס `PREPARING`

- **פרמטרים:** `pizza_type` (מחרוזת, חובה), `special_instructions` (מחרוזת, ברירת מחדל ריקה).
- **פעולה:** יצירת מזהה ייחודי (UUID), שמירת ההזמנה ב-MongoDB (סטטוס `PREPARING`), ושליחת הודעת JSON ל-Topic ב-Kafka שנקרא `pizza-orders`.
- משדר הודעה ל-Kafka לכל מטען בנפרד.

2. **`GET /order/{order_id}`**
   - יישום Cache-Aside: בדיקה ב-Redis (והחזרת `"source": "redis_cache"`), אם חסר - שליפה מ-Mongo ושמירה ב-Redis (והחזרת `"source": "mongodb"`).

שלב 3: פיתוח ה-Kitchen Worker

צרו קובץ `kitchen_worker.py`:

- מאזין ל-Kafka ל-`pizza-orders` תחת `'group_id='kitchen-team'`.
- עבור כל הודעה: ממתין 15 שניות (דימוי אבטחת המטען), מעדכן סטטוס ב-Mongo ל-`DELIVERED`, **ומוחק** את המפתח מ-Redis (Cache Invalidation).

שלב 4: פיתוח ה-Text Worker (התממה)

צרו קובץ `text_worker.py`:

- מאזין לאותו Topic תחת `'group_id='text-team'`.
- ניתוח טקסט של ה-`special_instructions`:
  - חיפוש מילות מפתח מזהירות ("allergy", "peanut", "gluten"). אם נמצאו -> עדכון שדה בוליאני **`allergies_flaged: true`**.
  - ניקוי טקסט (התממה): הסרת כל סימני הפיסוק, המרה לאותיות גדולות (Uppercase), ושמירה ב-Mongo תחת השדה **`cleaned_protocol`**.

קריטריונים להצלחה (Acceptance Criteria)

כדי לוודא שמערכת הסתרים שלכם עובדת, בצעו את הבדיקות הבאות דרך הטרמינל לאחר שהרצתם את הקונטיינרים:

שדרו את קובץ ה-JSON (המסווה כהזמנות פיצה):

```
\ "curl -X POST "http://localhost:8000/orders/batch
\ "H "Content-Type: application/json-
d @test_orders.json-
```

1.
2. בידקו את סטטוס המטען `order_1002` (שמכיל מילות מפתח על "אלרגיה לבוטנים"):
   - ב-15 השניות הראשונות: הסטטוס יהיה `PREPARING` והמידע יגיע מ-Mongo או Redis.
   - לאחר כ-20 שניות: המקור יחזור להיות Mongo (כי צוות הלוגיסטיקה מחק את ה-Cache). הסטטוס ישתנה ל-`DELIVERED`.
   - עליכם לראות ש-`tracker_suspected` הפך ל-`true`, ושההוראות נוקו מסימני הפיסוק בתוך `clean ed_protocol`.